



Adding Commands Objects

Extensibility

- Powershell is 'extensible'.
 - Extensible means we can “extend” the environment by adding extensions.
 - These extensions add new cmdlet and providers to the PS shell.
- (MSC snap-ins are, *by definition*, extensions of the MMC console.
 - The console itself serves as a platform to create a working tool by adding extensions.
- The difference is that MMC (without snap-ins) is completely useless.
- Powershell is fully functional and can be expanded by adding extensions and modules to it.

Two types of extensions: Snap-ins

- Snap-ins (*Deprecated in Powershell Core 7.x*)
 - Called “PSSnapin” (not the same as a MSC snap-in)
 - A few DLLs, and an XML file (for configuration)
 - Must be ‘registered’ with Powershell before they can be used.
 - MS is moving away from this extension model
- We can see the registered snap-in with the following command
 - `Get-PSSnapin -registered`
- Snap-ins also must be *loaded* to be available to the shell.
 - `Get-PSSnapin` will do that list any snap-ins that have been loaded.
 - `Add-PSSnapin` will load any PSSnapins that are available to powershell.

Two types of extensions: Snap-ins

- What happens when you install snap-in depends on the snap-in, but they will quite often add commands that were previously unavailable.
 - New cmdlets
 - New Providers
- Using `Get-Command -pssnapin <snapin>` we can see any commands that were installed as a result of installing the snapin.

Two types of extensions: Modules

- Installing modules is like installing Snap-ins, but it requires a bit more knowledge.
- **PSModulePath**: Powershell looks for modules (*autodiscovers*) in the PSModulePath.
 - The **PSModulePath** is an *environment variable* just like **Path** (the run path).
 - The variable holds the path where PS will look for modules.
 - Referencing the **Env:** provider, we can get the current configuration using **Get-Content Env:\PSModulePath**
- If you have other PSModulePaths, you can add them in advanced system information window (sysdm.cpl)
- **Get-Help** also references your module paths to find the help files of the cmdlet you're looking for.

Two types of extensions: Modules

- To show a list of installed Modules
`Get-Module`
- To add a Module (as long as its in the PSModule Path)
`Import-Module -name <moduleName>`
- To remove a Module
`Remove-Module -name <moduleName>`
- To Remove-All Modules
`Get-Module | Remove Module`
- Add all available modules to the current PS session
`Get-Module -ListAvailable -all | Get-Module`
- Using `Get-Command -module <module>` we can see any commands that were installed as a result of installing themodule.

Getting New Modules

- <https://powershellgallery.com>
 - An online Git-Hub repository of Powershell scripts and utilities.
 - Not MS verified, but are community created and managed.
- We can get these modules using the **PowerShellGet** module
 - Available in PS5.1 and up.
 - **\$PSVersionTable**
- Run **Register-PSRepository -default**
 - This will Psgallery as a repository (it's the default but you can register your own).
- **Unregister-PSRepository** will remove the reference to the repository.

Getting New Modules

- Use **Find-Module** to look for the Module you want (you can also just use the website).
 - **Find-Module *VMware***
- Once you found what you are looking for, use **Install-Module <modules name>**
- Remember your **\$PSModulesPath**. You may need to add the downloaded modules path to your EnvVariable in ordered for PS to see it.
- If the modules has a newer version, we can update it with **Update-Module**.

What do I do once I *have* a new Module

- Learn!
- Read the help!
- Do the examples!
- Search online!

Objects

Just data by another name

What are the things!?!

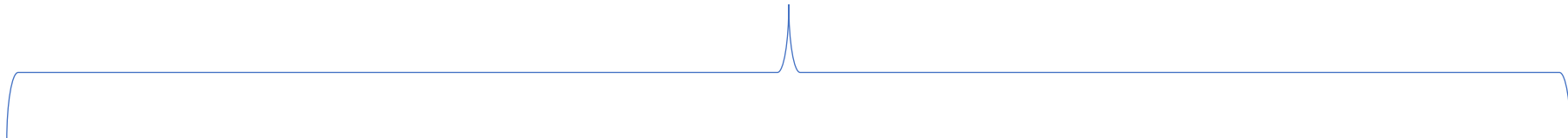
- As we saw with Exporting output to CSV, there is much more output, for a given command, that is show on the screen when we run that command.
- **Get-Process** (by default) shows seven columns.
 - Specified in the configuration files for the cmdlet.
- There are actually more than 60 columns of information but most of it is not shown on the screen.
- Easy way to see *all* of the data? Export the output of the command to a new Data Exchange format (CSV, XML, HTML).
- **Get-Process | ConvertTo-html | Out-File procs.html**

How the Data is organized.

- In addition to the information in each vertical column, each horizontal row has *actions* that can be performed against the data in that row.
 - Example for the ID column (which is the name of the process in number format) you can “kill” the process, or “refresh” its data.
- Essentially all data created by the output in a command is stored in a table, in memory.
- We can use that data as input for another command (Pipelining)
 - When we pipe, we pass the entire table to the next command for processing.

A new way to look at tables.

Collection



	Property 1	Property 2	Property 3	Property 4
Object 1	Values/Methods	Values/Methods	Values/Methods	Values/Methods
Object 2	Values/Methods	Values/Methods	Values/Methods	Values/Methods
Object 3	Values/Methods	Values/Methods	Values/Methods	Values/Methods
Object 4	Values/Methods	Values/Methods	Values/Methods	Values/Methods

Why use this data/object format?

- Object-oriented programming languages (OOP) is quite common.
 - Windows is object-oriented
 - Most applications that *run* on Windows are also object-oriented.
- By using the same data structures as Windows, the data being used translates easily between applications or commands being used in that environment (think: no data type conversions).
- Using objects is efficient, easy, and flexible
- Parsing text, multiple times per command an alternative, and lack of flexibility, it's the alternative (unix shells, Perl, etc)
 - Requires *a lot* of practices with Regular Expressions (RegEx).
- Unix is a Text-Based OS. Windows is an Object-Based OS.

Discovering Objects

- `Get-help`'s sibling: `Get-Member`
 - Help is good for background information, examples and syntax.
- `Get-member` (GM)...
 - ...is *pipelined* to (like `Format-Table` or `Format-List`)
 - ...will produce a *collection* of *objects*
 - ...has types of properties (we call these `MemberTypes` or `Members`)
 - ...will display the commands *typeName*.
 - TypeName is the type of data being passed (more in chapter 9)

MemberTypes (Properties/Methods/Events)

- Common MemberType Properties
 - AliasProperty
 - Method
 - Event
 - NoteProperty
 - Property
 - PropertySet
 - ScriptProperty
- Due to PS's extensibility, property types are added all the time.
- Most of them can be thought of as the same thing: [a property](#).

Properties

- Properties always contain a **Value**.
 - ProcessID (ID) might be 39493 (the PID)
 - Name could be Notepad.exe, or Explorer.exe
- Properties describe something about the object.
 - And are usually 'Read-only' (but not always).

Methods

- Most CMDLETs support **Methods** (not all though)
- **Methods** are actions that an object can *do*.
- Most methods are built into the functionality of the cmdlet.
- Using methods can take a lot of extra training and experience (in .Net programming) to fully understand how to use them.
- You *could* spend time finding how to use the 'kill' method of **Get-Process**, or you could simply use the **Stop-Process** command, instead.

Methods

Consider

- Killing a process example 1:
 - `Get-process -name Notepad.exe | Stop-Process`
- Killing a process example 1:
 - `Stop-Process -name Notepad.exe`
- Killing a process example 1:
 - `$proc = Get-Process -name notepad`
 - `$proc.kill()`

Events

- Events are the Objects way of sending the console notifications.
- Outside of the scope of this course.

Sorting-Objects

- Sometimes the default format of a commands output is not what we need.
- **Sort-Object** will allow you to Sort the order in which objects are displayed
(remember that an object is a row in a table)
 - **-property** (positional) selects the property to sort the table by
 - **-descending** changes the default sorting order (ascending)
- We can sort by multiple values as well
(remember the values are the *cell* of the table; the intersection of the column (property) and row (object))
 - Example: **GCI | sort -property Length,Name**
 - Sort by length first; if two values are equal, then sort by the name value.

Select-Object

- **select-object** allows you to specify the properties to display in your output
 - **-property** The properties to be selected (string array)
 - **-First #** Select the first 10 objects
 - **-Last #** select the last 10 objects
- **select-object** can:
 - add properties that aren't there in the original output
 - remove properties that are irrelevant
 - filter your output
 - order the selected columns based on the order in which they are parsed
- **gps | select NAME,ID,CPU** VS **gps | select ID,CPU,Name**

PSObject

- When running a **Select-Object** command, the resulting output object type, passed in the pipeline, is changed to a **PSObject**.
 - PSObject is a *custom* object, which makes sense because you are formatting a regular object in a custom way.
 - When using a PSObject, Powershell ignores the rules for formatting the original object and uses the properties of a PSObject instead.

Select-Object in a nutshell

- New Powershellers confuse `where-Object` with `Select-Object`.
- Clarification:
 - `Select-object` is for formatting your table.
 - Choose the properties (`-property`) you want, and order them (to create columns)
 - Display `X` number of rows where `X = -First X` or `-Last X`
- `where-Object` (covered later) filters objects in or out of the pipeline.

Points to Remember

- PS helpfiles do not contain Property information about an object. You'll need to pipe the objects to **Get-Member** to see a list of properties.
- If a command does not produce output, **Get-Member** may not show anything about that command.
- Put a space on either side of the pipe to keep things clean.
 - **Get-Process | GM**
 - Not **Get-Process|GM**
- Focus on trying to intuit on what is in the pipeline and how it can be used.